Chapter 2. Foundations of logic

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 Propositions, syntax, and valuations

Propositional variables p, q take values in $\{T, F\}$. A valuation $v : \{p, q, \ldots\} \to \{T, F\}$ determines a formula's truth recursively. Well-formed syntax comes first. Parentheses or a binding convention must fix the structure of `p → q ∧ r`.

Successive implications associate to the right. Still parenthesize complicated formulas. The implication $p \to q$ is false only for $p = T, q = F$. Thus "if it rains, the ground is wet" does not promise dry ground without rain.

Natural-language conditionals may suggest causation. Propositional logic retains only their truth tables; causal claims require another model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1.1 Evaluating a syntax tree

List all atomic propositions and fix parentheses. For

$$\varphi = \neg p \lor (q \land r),$$

the root is $\lor$, with subtrees $\neg p$ and $q \land r$. Use precedence

$$\neg > \land > \lor > \rightarrow > \leftrightarrow .$$

Explicit parentheses still prevent visual ambiguity. For $v(p) = F, v(q) = T, v(r) = F$, evaluate inside out:

$$v(\neg p) = T, \qquad v(q \land r) = T \land F = F,$$
$$v(\varphi) = T \lor F = T.$$

A formula with n atoms has 2^n truth-table rows. Each row assigns every atom, then evaluates the tree. The same syntax accepts different valuations. Changing parentheses changes the tree and may change the formula despite similar text.

2.2 Truth conditions of five connectives

For every valuation,

p	q	$p \land q$	$p \lor q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
T	T	T	T	T	T	F
T	F	F	T	F	F	T
F	T	F	T	T	F	T
F	F	F	F	T	T	F

Negation $\neg p$ reverses truth. Inclusive or is $\lor$; exclusive or is $\oplus$. Natural-language "or" may be ambiguous, so mathematical notation must specify which one. For implication, identify its unique false case before filling the other three.

2.3 Tautologies, equivalence, and counterexamples

A formula φ is a tautology if every valuation makes it true, and satisfiable if at least one does. These quantifiers differ. Define $\varphi \equiv \psi$ to mean equal truth values under every valuation. Check equivalence with a full truth table or truth-condition rewrites for arbitrary valuations:

$$p \to q \equiv \neg p \lor q \equiv \neg(p \land \neg q).$$

One valuation with different truth values refutes an equivalence claim.

The figure gives criteria for tautology, contradiction, and contingency on the left, and compares three formulas under the same p, q valuations on the right. Hide the last column, evaluate the rows, then check the classification.

2.3 Tautologies, equivalence, and counterexamples (continued)

Three proposition classes

Tautology: every row true

Contradiction: every row false

Contingency: true and false rows

Check every truth assignment

Check formulas in one table

p	q	p→q	p∧¬p	p∨q
T	T	T	F	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

p→q and p∨q are contingencies; p∧¬p is a contradiction

Satisfiability is weaker than tautology. The formula $p \land q$ is satisfiable but false when $v(p) = F$.

2.4 Logical laws and algebraic rewrites

Common laws include double negation, De Morgan, distribution, absorption, idempotence, commutativity, and associativity:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q, \qquad p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r).$$

These are tautology schemata. State the schema and its location at every rewrite. The false rewrite $p \vee (q \wedge r) \equiv (p \vee q) \wedge r$ is refuted by $p = T, q = F, r = F$.

Truth tables are finite checks. Algebraic rewrites preserve visible structure but need a valid reason at every step.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.4.1 Duality, substitution, and traceable simplification

For formulas using $\wedge, \vee, T_0, F_0$, form the dual by interchanging $\wedge, \vee$ and T_0, F_0, leaving atoms fixed. For example,

$$P = (p \vee q) \wedge T_0, \qquad P^d = (p \wedge q) \vee F_0.$$

The duality theorem says a tautology $P \Leftrightarrow Q$ gives a tautology $P^d \Leftrightarrow Q^d$. Duality differs from complementation, which also negates literals.

For a structural-induction proof, reverse every atom in a valuation v to obtain v^c. Then

$$\neg P(v) = P^d(v^c).$$

Atoms and constants satisfy this directly. For a conjunction, the induction hypothesis and De Morgan give

$$\neg(A \wedge B) = \neg A \vee \neg B = A^d(v^c) \vee B^d(v^c),$$

which evaluates its dual. The disjunction case is symmetric. If $P(v) = Q(v)$ for every v, negate both sides and let $w = v^c$ range over all valuations to get $P^d(w) = Q^d(w)$.

2.4.1 Duality, substitution, and traceable simplification (continued)

The substitution theorem permits consistent replacement of atoms in a tautology by compound formulas. In $p \to q \Leftrightarrow \neg p \lor q$, replace p by $r \lor s$ and q by $t \land u$. If $P \Leftrightarrow Q$, one or more occurrences of P inside another formula can likewise be replaced by Q.

To justify substitution, treat the compound formulas' truth values as the original atoms' valuation. The tautology holds at that valuation; consistent substitution preserves the truth relationships at all positions.

For example,

$$
\begin{aligned}
& \neg((p \lor q) \land r) \lor \neg q \\
\Leftrightarrow & (\neg(p \lor q) \lor \neg r) \lor \neg q && \text{De Morgan} \\
\Leftrightarrow & ((\neg p \land \neg q) \lor \neg r) \lor \neg q && \text{De Morgan} \\
\Leftrightarrow & \neg q \lor \neg r && \text{commutativity, associativity, absorption.}
\end{aligned}
$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.4.1 Duality, substitution, and traceable simplification (continued)

For the last step, rearrange to $(\neg q \lor (\neg p \land \neg q)) \lor \neg r$, then absorb $\neg p \land \neg q$ into $\neg q$. Match each main rewrite to its reason. Absorption does not allow arbitrary deletion; the required subformula structure must occur. Double negation, distribution, and absorption are distinct laws.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 Implication, converse, and inferential validity

The converse of $p \to q$ is $q \to p$, and its contrapositive is $\neg q \to \neg p$. Only the contrapositive is necessarily equivalent.

An argument $\Gamma \therefore \varphi$ is valid if every valuation making all premises Γ true also makes φ true. Modus ponens is

$$p, \quad p \to q \quad \therefore q,$$

with tautological conditional form

$$\big(p \land (p \to q)\big) \to q.$$

If both premises hold, $p = T$ and $p \to q = T$, so q cannot be false. No truth-table row has all premises true and conclusion false.

Affirming the consequent,

$$q, \quad p \to q \quad \therefore p,$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 Implication, converse, and inferential validity (continued)

is invalid. The assignment $p = F, q = T$ makes its premises true and conclusion false. Truth of premises and validity concern different levels; true premises can still occur in an unsound argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 Inference rules and proof structure

Besides modus ponens, common rules include modus tollens, hypothetical syllogism, disjunctive syllogism, and resolution. Resolving clauses $(p \lor A), (\neg p \lor B)$ gives $A \lor B$. Each row below expresses a tautological implication under arbitrary substitution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 Inference rules and proof structure (continued)

Rule	Premises	Conclusion
Modus ponens	$P, P \to Q$	Q
Modus tollens	$\neg Q, P \to Q$	$\neg P$
Hypothetical syllogism	$P \to Q, Q \to R$	$P \to R$
Disjunctive syllogism	$P \lor Q, \neg P$	Q
Conjunction introduction	P, Q	$P \land Q$
Simplification	$P \land Q$	P
Addition	P	$P \lor Q$
Resolution	$P \lor A, \neg P \lor B$	$A \lor B$
Constructive dilemma	$P \to Q, R \to S, P \lor R$	$Q \lor S$
Destructive dilemma	$P \to Q, R \to S, \neg Q \lor \neg S$	$\neg P \lor \neg R$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 Inference rules and proof structure (continued)

For $p \to q, q \to r, \neg r, s \lor p \therefore s$, the full derivation is:

Step	Statement	Reason
1	$p \to q$	Premise
2	$q \to r$	Premise
3	$\neg r$	Premise
4	$s \lor p$	Premise
5	$\neg q$	Modus tollens, 2 and 3
6	$\neg p$	Modus tollens, 1 and 5
7	s	Disjunctive syllogism, 4 and 6

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.6 Inference rules and proof structure (continued)

Counterexample checking also belongs to proof assessment. The invalid $p \to q, q \therefore p$ fails at $p = F, q = T$. Validity differs from factual truth of premises. Do not reverse an implication without justification.

An inference proof is a finite sequence of premises, substitution instances of known tautologies, and rule applications to earlier lines. Treating every implication as a biconditional is a common mistake. Larger arguments can eliminate implications, convert to CNF, and apply resolution, connecting to Chapter 15.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.7 Full quantifier forms

For a nonempty domain D, or an explicitly stated alternative convention,

$$(\forall x \in D) P(x) \iff \forall x\,(x \in D \to P(x)),$$
$$(\exists x \in D) P(x) \iff \exists x\,(x \in D \wedge P(x)).$$

Negation gives

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \qquad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

Existence requires a witness; a universal proof handles an arbitrary fixed element.

Distinguish open and closed statements. Over integers, $p(x) : x + 2 \text{ is even}$ has free variable x, so is not itself a proposition. Substituting three makes it false and four makes it true. In $\forall x\, p(x)$, the quantifier binds x, producing a closed statement. Declare the universe before writing $\exists x P(x)$; changing integers to positive integers or reals may change a statement's truth.

Implicit quantifiers should be restored in proofs. "The sum of two odd integers is even" means

$$\forall m \forall n\,(m, n \text{ odd} \to m + n \text{ even}).$$

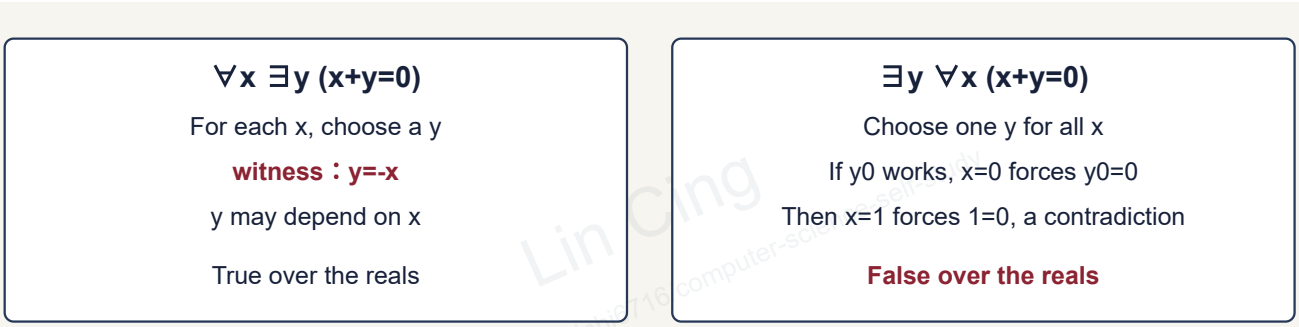
2.7 Full quantifier forms (continued)

This prevents substituting one example for a general argument. Bounded universals use implication and bounded existentials conjunction; exchanging them changes truth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.8 Nested quantifiers and order

Over reals, $\forall x \exists y (y > x)$ is true, permitting $y = x + 1$, whereas $\exists y \forall x (y > x)$ is false. The first allows dependence on x; the second requires a common witness.



Likewise $\forall x \exists y (x + y = 0)$ uses $y = -x$. A fixed y in $\exists y \forall x (x + y = 0)$ must equal zero at $x = 0$ and minus one at $x = 1$, an impossibility.

2.8 Nested quantifiers and order (continued)

Quantifiers of the same kind commute:

$$\forall x \forall y P(x, y) \Leftrightarrow \forall y \forall x P(x, y),$$
$$\exists x \exists y P(x, y) \Leftrightarrow \exists y \exists x P(x, y).$$

Mixed quantifiers do not generally commute. One valid direction is

$$\exists x \forall y P(x, y) \Rightarrow \forall y \exists x P(x, y).$$

A common witness works separately for each y; separately available witnesses need not coincide. Quantification relates closely to selecting functions. A pairing table illustrates dependence on finite domains.

2.8.1 Quantifier rules and side conditions

Do not merely erase quantifiers. Explain how elements are obtained.

Rule	Form	Condition
Universal specification	$\forall x P(x) \therefore P(c)$	c denotes an element of the domain
Universal generalization	$P(c) \therefore \forall x P(x)$	c is arbitrary and fixed; no special property is used
Existential specification or elimination	Temporarily assume $P(c)$ from $\exists x P(x)$ in a subproof	c is fresh and cannot remain free in the final conclusion
Existential generalization	$P(c) \therefore \exists x P(x)$	c is a domain element or specified witness

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.8.1 Quantifier rules and side conditions (continued)

Arbitrary but fixed does not mean selecting a special object and generalizing it. An arbitrary triangle T cannot additionally be assumed equilateral. Universal generalization requires that c not occur free in any undischarged assumption on which the derivation depends. Assuming $P(c)$ alone does not license $\forall x P(x)$.

For existential elimination, derive C under temporary $P(c)$ with fresh c. Discharge it only if c occurs free neither in C nor in any other undischarged assumption. One cannot export $P(c)$ as a proved fact about a specified external constant.

The premises $\exists x P(x), \exists x Q(x)$ cannot share one fresh name to conclude $\exists x (P(x) \land Q(x))$. Over integers, evenness and oddness each have witnesses but no common one. Use separate fresh names unless equality of witnesses is established.

For example,

$$\forall x (P(x) \to Q(x)), P(a) \therefore Q(a).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.8.1 Quantifier rules and side conditions (continued)

Universal specification gives $P(a) \to Q(a)$, then modus ponens gives $Q(a)$. In contrast, all squares are rectangles and r is a rectangle do not imply r is a square:

$$\forall x(\mathrm{Square}(x) \to \mathrm{Rectangle}(x)), \quad \mathrm{Rectangle}(r).$$

Inferring $\mathrm{Square}(r)$ affirms the consequent inside quantified notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9 Uniqueness, vacuous truth, and boundaries

Expand unique existence as

$$\exists! x P(x) \iff \exists x \big(P(x) \land \forall y (P(y) \to y = x)\big).$$

If no domain element satisfies P, the statement $\forall x(P(x) \to Q(x))$ remains true vacuously. It asserts no existence. Every element of the empty set satisfies any specified predicate vacuously.

Definitions often use "if and only if." Defining even n by $n = 2k$ for some integer k gives two descriptions of one definition. For a claim $x \in A \iff x \in B$, however, prove both membership implications. Obtain

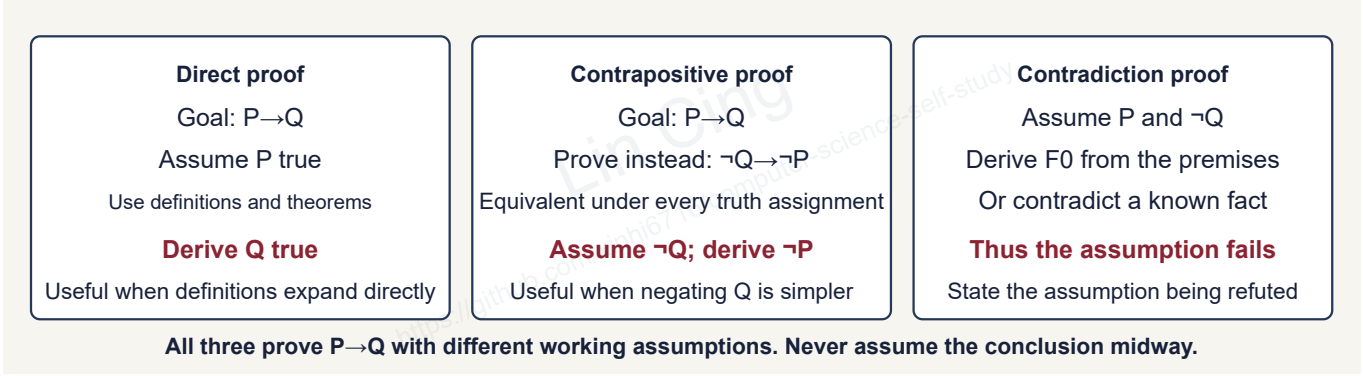
$$\forall x \, (x \in A \leftrightarrow x \in B)$$

and apply extensionality to conclude $A = B$. One direction ordinarily gives only one inclusion. Uniqueness needs both existence and at most one solution; equality of two hypothetical solutions proves only the latter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.10 Direct, contrapositive, and contradiction proofs

To prove $p \to q$, a direct proof assumes p and derives q; a contrapositive proof assumes $\neg q$ and derives $\neg p$; a contradiction proof assumes $p \wedge \neg q$ and derives an impossibility. All suffice logically, though their difficulty differs. Track active assumptions throughout and never silently assume the desired conclusion.



Consider

2.10 Direct, contrapositive, and contradiction proofs (continued)

$$\forall m \in \mathbb{Z}\,(m^2 \text{ even} \to m \text{ even}).$$

Fix an arbitrary integer m and prove its contrapositive. If m is not even, it is odd, so $m = 2k + 1$ for an integer k. Then

$$m^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

The expression in parentheses is an integer, so m^2 is odd. This proves the universal odd-square implication; contraposition proves the theorem.

For a direct proof, even m, n have $m = 2a, n = 2b$ for integers a, b. Hence

$$m + n = 2a + 2b = 2(a + b)$$

is even. Universal generalization yields the claim for all integer pairs. Name witnesses and state algebraic steps precisely enough for independent checking.

2.11 Cases, existence, and counterexamples

A case proof must cover all cases allowed by the hypotheses and prove each. Cases may overlap, provided none is omitted; residue classes are naturally disjoint. One counterexample refutes a universal claim. Refuting existence requires proving no witness exists, often by universal negation.

"Every integer above one is odd or prime" fails at four, which is neither. Nine is odd, so it satisfies that disjunction despite being composite. It refutes the different claim "every odd integer above one is prime." Formalize first.

To prove $n^2 + n$ even for every integer, use exhaustive parity cases:

1. If $n = 2k$, then $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$.
2. If $n = 2k + 1$, then $n^2 + n = (4k^2 + 4k + 1) + (2k + 1) = 2(2k^2 + 3k + 1)$.

Both parentheses contain integers. The cases cover every integer, so universal generalization finishes the proof.

A constructive proof of $\exists n \in \mathbb{Z}, n^2 = n + 2$ supplies $n = 2$, since $2^2 = 2 + 2$. An existence proof by contradiction instead negates the quantified assertion and derives a contradiction.

2.11 Cases, existence, and counterexamples (continued)

For the equivalence $n \text{ even} \Leftrightarrow n^2 \text{ even}$, the forward direction uses $n = 2k$, giving $n^2 = 2(2k^2)$. The reverse uses the earlier odd-square contrapositive. Only after both directions may one write the biconditional. A counterexample must meet every hypothesis and violate the conclusion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.12 Formalizing the theorem

Before proving a statement, specify its domain and variable types, hidden hypotheses such as nonzero inputs or connectivity, its logical form, both directions when required, and counterexample or empty-set boundaries.

For example, $0 < b \leq a$ implies $\gcd(a, b) = \gcd(a - b, b)$. One numerical check is not a proof. Check whether $a - b$ stays positive, or use integer division to avoid repeated positivity issues. Exercises and full solutions are separate PDFs. Chapter 4 applies divisibility and proof methods to number theory.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.13 Constructing DNF and CNF from any truth table

Fix atoms $p_1, \ldots, p_n$. For each valuation v making φ true, form the minterm

$$m_v = \bigwedge_{i=1}^{n} \begin{cases} p_i, & v(p_i) = T, \\ \neg p_i, & v(p_i) = F. \end{cases}$$

Each literal forces its coordinate, so m_v is true exactly at v. Thus at any valuation w, the disjunction $\bigvee_{v:\varphi(v)=T} m_v$ is true exactly when w is a true row of φ. This proves canonical DNF. An empty disjunction is F.

For each false row v, take $c_v = \neg m_v$, the disjunction of reversed literals. It is false only at v. Their conjunction $\bigwedge_{v:\varphi(v)=F} c_v$ excludes precisely all false rows, giving canonical CNF. An empty conjunction is T.

For $\varphi = (p \leftrightarrow q) \vee r$, valuations 000 through 111 give T, T, F, T, F, T, T, T. False rows 010 and 100 give

$$\varphi \equiv (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.13 Constructing DNF and CNF from any truth table (continued)

Its six true-row minterms simplify to $(\neg p \wedge \neg q) \vee (p \wedge q) \vee r$. Normal forms guarantee existence, rather than minimal terms or gates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.14 Functional completeness of NAND

Define $N(p, q) = \neg(p \wedge q)$. Truth conditions give

$$\neg p = N(p, p), \qquad p \wedge q = N(N(p, q), N(p, q)),$$
$$p \vee q = N(N(p, p), N(q, q)).$$

The first uses idempotence, the second negates NAND, and the third uses De Morgan.

Every finite-variable Boolean function has a DNF using AND, OR, and NOT. Replace each operation bottom-up in its syntax tree by these NAND constructions. Structural induction preserves the function. This proves general representability, beyond checking three particular functions.

With at least one input p, constant $T = N(p, N(p, p))$ and $F = N(T, T)$ cover constant functions. With neither inputs nor constant sources, a binary gate cannot start a circuit; a constant input is required. Chapter 15 connects these equivalences with physical gates.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study